

Artificial Analysis Intelligence Index Evaluations

Evaluations that make up the current Artificial Analysis Intelligence Index, grouped by capability.

Agents

AA-Briefcase v1.1

- Status: Included in Artificial Analysis Intelligence Index v4.3.2 at 15% weighting.
- Description: AA-Briefcase is a new benchmark for testing models on realistic knowledge work tasks in complex projects built by industry experts. Models are evaluated on multi-week knowledge work projects, each with many linked tasks and thousands of input source files. AA-Briefcase combines rubric and pairwise grading to evaluate verifiable task success, analytical quality, and presentation quality, giving a holistic view of overall agentic capability in knowledge work.
- Changes from AA-Briefcase v1: v1.1 changes only how Elo ratings are fitted. We fit the ratings with a [Crowd-BT](#) model. For a comparison of submissions
 - i
 - i and
 - j
 - j with fitted strengths
 - θ_i
 - θ
 - i
 -
 - and
 - θ_j
 - θ
 - j
 -
 - , and annotator quality
 - η_k
 - η
 - k
 -
 - :
 - $p = \eta_k \sigma(\theta_i - \theta_j) + (1 - \eta_k)(1 - \sigma(\theta_i - \theta_j))$
 - $p = \eta$
 - k
 -
 - $\sigma(\theta$
 - i

-
- $-\theta$
- j
-
- $)+(1-\eta$
- k
-
- $)(1-\sigma(\theta$
- i
-
- $-\theta$
- j
-

•))

- We define
- $\varepsilon=1-\eta k$
- $\varepsilon=1-\eta$
- k
-
- , giving:
- $p=\varepsilon+(1-2\varepsilon) \sigma(\theta_i-\theta_j)$
- $p=\varepsilon+(1-2\varepsilon)\sigma(\theta$
- i
-
- $-\theta$
- j
-

•)

- where
- ε
- ε is fitted per scope on past AA-Briefcase judgements. Rubric grading is decided by a deterministic comparison rather than a judge, so
- $\varepsilon=0$
- $\varepsilon=0$ for that scope. Whilst Elo scores shift, rank ordering is largely preserved.
- Example dataset: <https://huggingface.co/datasets/ArtificialAnalysis/AA-Briefcase-Lite>
- Agent harness: <https://github.com/ArtificialAnalysis/Stirrup>
- Implementation:
 - Each AA-Briefcase scenario is a realistic multi-week business problem, organized as a multi-week workflow that the agent works through in sequence, with 2-5 tasks per week. Although tasks within a scenario share files and context across weeks, models currently complete each task in an independent run, without carrying over their own prior submissions. The agent receives the task description and accessible source

files, then produces final deliverable files without live interaction or iterative feedback during execution.

- Scenario source pools include shared files and week-specific files, mixing real, augmented, and synthetic materials. Source files are designed to include realistic professional artifacts such as Slack exports, spreadsheets, PDFs, interview transcripts, market research, standards documents, app-store pages, board materials, emails, and other business records. Later-week tasks may receive standardized base-case files (the same reference work products given to every model), so each task stays independently runnable while preserving continuity across the week.
- Model submissions are run with [Stirrup](#) in a week-scoped E2B sandbox.
 - Turns: Agents run for up to 500 turns per task.
 - Tools: The agent is given a single code-execution tool that runs shell commands and code inside the sandbox, plus the finish tools below (and a view-image tool when the model supports vision). The sandbox has no internet access, so the agent can only use the provided source files.
 - Sandbox: Each scenario/week sandbox is built from that week's source files, with standard Python packages and system tools for document processing and scientific computing pre-installed.
 - Finish tools: A finish tool, which the agent calls to submit a summary and the absolute paths of its deliverables (validated to be actual files, not directories or missing paths), and an `abandon_task_finish` (give-up) tool, which it calls with a reason only when it concludes the task is genuinely impossible.
- Prompts: The prompts used across generation and grading:

Agent system prompt:

You are an AI agent working on a specific task within a multi-week simulated workplace scenario. Each task is part of a longer workflow; your job is to complete the current task using the tools provided in up to 500 steps, then submit your deliverables.

When you are done you must call the ``finish`` tool as your final step, passing a brief summary of what you accomplished and a list of absolute paths for every deliverable file.

If you have genuinely concluded that the task cannot be completed – for example because required inputs are missing, a hard dependency is unavailable, or the request itself is incoherent – call the ``abandon_task_finish`` tool with a brief reason instead. Do not use it to escape difficulty.

- You cannot interact with the user during the task. Record any clarifying assumptions you made in your finish summary.

Agent task prompt:

<execution_context>

Sandbox

You operate inside an isolated Linux container through the `code_exec` tool, which runs shell commands and lets you read, create, and edit files. Commands run as the unprivileged user `user` (UID 1000), starting from `/home/user`. Passwordless `sudo` exists but is rarely needed, since your home directory is fully writable.

Every command runs independently: no working directory, environment variable, or other shell state carries over from one call to the next. Prefer absolute paths for both files and commands, and do not navigate with `cd` across calls – a `cd` in one command is gone by the next, so relying on it leaves you silently operating in the wrong place. When a step genuinely needs a different directory, chain it into the same command (e.g. `cd /home/user/work && python build.py`).

No network

The container has no outbound connectivity, and there is no proxy, allowlist, or flag that can turn it on – treat the environment as permanently offline. Anything that reaches for the internet will fail, including package installs (`pip`, `npm`, `apt`), remote `git` operations, and any HTTP/HTTPS client request from any language.

Identify a network block by its error signature rather than by guessing: failed name resolution (`Could not resolve host`, `Temporary failure in name resolution`), an unreachable route (`Network is unreachable`, a refused or timed-out connection to a public host), or a stalled TLS handshake. When you see these, the failure is structural – do not retry the same call and do not hunt for a workaround (mirrors, alternate hosts, cached copies). Re-plan using only what is already installed and what ships inside your workspace.

Filesystem

- Writable: everything under `/home/user/` plus `/tmp`. Use these for deliverables, intermediate files, and caches.
- Read-only inputs:
 - `/home/user/shared/` – reference material shared across the whole scenario
 - `/home/user/week/` – documents specific to this week's tasks

Copy these into a working folder before transforming them rather than editing them in place.

Runtime

A broad scientific-computing and document-processing stack is already installed, so confirm what is present before assuming a gap:

- Python 3.13 with the usual data stack (numpy, pandas, polars, scipy), plotting (matplotlib, plotly), the scikit-learn ML family, and document tooling (python-docx, python-pptx, openpyxl, PyMuPDF, pdfplumber, reportlab, weasyprint, Pillow, opencv), plus Playwright.
- System tools include LibreOffice, Pandoc, Tesseract, FFmpeg, ImageMagick, Ghostscript, TeX Live, OpenJDK, Chromium, jq, and git.
- Check availability with `pip show <pkg>` or `which <tool>` instead of installing – installs fail offline, but almost anything you would reach for is already here.
- matplotlib runs headless (`MPLBACKEND=Agg`): write figures to files; never call `plt.show()`.
- Commands are terminated after 20 minutes. Keep them bounded, persist intermediate results to disk, and split long jobs into smaller steps.

Submitting your work

Finish by calling the `finish` tool – anything not submitted through it is not graded. Your call must include:

1. A short summary of what you accomplished.
2. Absolute paths to every deliverable (files only, not folders).

Save each deliverable directly in `/home/user` under the exact filename the task asks for – not in a subdirectory.

Save deliverables as ordinary, visible files. Do not leave the only copy of your work in a dot-prefixed file or directory (e.g. `.submission.txt`, `.outputs/report.md`), including inside an archive; a `.zip` is fine when the task explicitly asks for one. Assume your files will be opened and edited by others after submission, so write them to last.

If the task genuinely cannot be completed, call the `abandon\_task\_finish` tool with a brief reason instead. Use it only when you have concluded the work is impossible – not to escape a difficult task.

</execution_context>

<scenario_overview>
{scenario_overview}
</scenario_overview>

<week_overview>
{week_overview}
</week_overview>

<task_description>

```
{task}  
</task_description>
```

```
<deliverables>  
Submit these files, by exact name, saved directly in `/home/user`:  
{expected_output_filenames}  
</deliverables>
```

- Please begin working on the task now.

Binary rubric grading prompt:

You are grading a submitted deliverable against one binary rubric check.

The user message contains:

- the task instructions,
- the rubric item,
- the submitted artifact content.

Submitted artifacts may appear as text blocks, image blocks, or parser notes for unsupported content.

Use only evidence from the submitted artifact content. Do not infer facts from filenames, task instructions, or rubric text unless the submitted artifact content supports them.

Beyond the task instructions and rubric in the user message, you only ever receive the submitted artifact itself, never the external source files it cites. Do not fail an item merely because you cannot open or cross-check a cited source – judge citations on whether they are present, specific, and well-formed in the submission, not on whether the source's contents can be independently confirmed.

Return a strict binary judgment:

- passed=true only if the pass criteria are satisfied.
- passed=false if any required element is missing, materially wrong, unsupported, or not evidenced.

Write concise reasoning that cites submitted artifact evidence or the absence of evidence.

- Do not award partial credit.
- Each task is graded against two styles of checks. Rubric checks are binary pass/fail criteria scored against a single submission. Pairwise checks compare two submissions for the same task and return a preferred submission or tie. There are

two kinds: *Analytical Quality* (which output has deeper, better-structured analysis) and *Presentation* (which output is more professionally presented).

- Rubric grading and the analytical-quality and presentation pairwise comparisons each use a panel of three judges rather than a single judge, reducing bias toward submissions from the same model or model family. Rubric grading uses Claude Opus 4.8 at max effort, GPT-5.5 at high reasoning, and Gemini 3.1 Pro Preview at high reasoning. Pairwise comparisons use Claude Opus 5 at high effort, GPT-5.6 Sol at medium reasoning, and Gemini 3.8 Flash at high reasoning. Each rubric verdict and LLM-judged pairwise comparison is decided by one judge sampled from its panel, with sampling balanced across checks and matches. To keep results comparable, a given rubric check is always graded by the same judge. AA-Briefcase Elo is the headline metric of this evaluation: it aggregates analytical quality Elo, presentation Elo, and rubric pass rate, with rubric performance converted into Elo via synthetic head-to-head matches using a maximum-likelihood Elo aggregation. Each scope is fitted with a [Crowd-BT](#) model.
- Intelligence Index Integration: For inclusion in the Intelligence Index, the combined AA-Briefcase v1.1 Elo is frozen at the time of a model's addition and normalized as $\text{clamp}((\text{Elo} - 500) / 2000)$, the same mapping applied to GDPval-AA v2.1. The Elo scale is anchored to GPT-5.5 (medium) at 1000, while the fixed normalization range preserves stable Intelligence Index contributions over time. Artificial Analysis may update the reference parameters as models progress against the evaluation, to maintain meaningful differentiation in the Intelligence Index.

[GDPval-AA v2.1](#)

- Description: GDPval-AA v2.1 is Artificial Analysis' evaluation framework for OpenAI's GDPval dataset. It assesses language models' capabilities on economically valuable tasks, covering 44 occupations across key sectors contributing to GDP in the United States.
- Changes from GDPval-AA v2: v2.1 changes only how the Elo scale is fixed:
 - We anchor the scale by pinning DeepSeek V4.1 Flash (max) at 1600.
 - We fit the ratings with a [Crowd-BT](#) model. For a comparison of submissions
 - i
 - i and
 - j
 - j with fitted strengths
 - θ_i
 - θ
 - i
 -
 - and
 - θ_j
 - θ
 - j
 -

- , and annotator quality
- η_k
- η
- k
-
- :
- $p = \eta_k \sigma(\theta_i - \theta_j) + (1 - \eta_k)(1 - \sigma(\theta_i - \theta_j))$
- $p = \eta$
- k
-
- $\sigma(\theta$
- i
-
- $-\theta$
- j
-
- $)+(1 - \eta$
- k
-
- $)(1 - \sigma(\theta$
- i
-
- $-\theta$
- j
-
-))
- We define
- $\varepsilon = 1 - \eta_k$
- $\varepsilon = 1 - \eta$
- k
-
- , giving:
- $p = \varepsilon + (1 - 2\varepsilon) \sigma(\theta_i - \theta_j)$
- $p = \varepsilon + (1 - 2\varepsilon) \sigma(\theta$
- i
-
- $-\theta$
- j
-
-)
- where
- ε

- ε is fitted on past GDPval-AA judgements.
 - Whilst Elo scores shift, rank ordering is largely preserved.
- Paper: <https://arxiv.org/abs/2510.04374>
- Agent harness: <https://github.com/ArtificialAnalysis/Stirrup>
- Dataset:
 - We base our evaluation on the public gold OpenAI GDPval dataset from <https://huggingface.co/datasets/openai/gdpval>
 - Some Microsoft Office files in the dataset had missing metadata parts or malformed relationship entries that prevented LibreOffice from opening them. We added the minimal missing metadata and fixed the malformed entries to ensure compatibility. Document body, slide content, and layout were not changed.
- Implementation: This evaluation comprises two stages:
 - Task Submission – Models are given a task and required to produce one or more files.
 - Pairwise Grading – A judge sampled from a panel of three frontier LLM judges blindly ranks two submissions for the same task, each created by a different model.
 - Elo Calculation: After collecting pairwise rankings, we fit them to a [Crowd-BT](#) model via maximum likelihood estimation, and compute confidence intervals using the sandwich estimator to establish our final Elo metric. We anchor the Elo scale by pinning DeepSeek V4.1 Flash (max) at 1600. Every other rating is fitted relative to that anchor.
 - Intelligence Index Integration: For inclusion in the Intelligence Index, GDPval-AA v2.1 Elo scores are frozen at the time of a model's addition and normalized as $\text{clamp}((\text{Elo} - 500) / 2000)$ for inclusion in the Intelligence Index. The v2.1 Elo scale is anchored to DeepSeek V4.1 Flash (max) at 1600, while the fixed normalization range preserves stable Intelligence Index contributions over time. Artificial Analysis may update the reference parameters as models progress against the evaluation, to maintain meaningful differentiation in the Intelligence Index.
- Task Submission Details:
 - All models are run using our open source agentic harness, [Stirrup](#). Within the harness, models are given a code execution environment (E2B sandbox), and the following six tools to call at their discretion:
 - Web Fetch – Fetches and extracts main content from a web page as markdown.
 - Web Search – Searches the web using Brave Search API; returns the top 5 results with title, URL, and description.
 - View Image – Reads and displays image files (.png, .jpg, .jpeg) from the sandbox as native image tokens for LLM consumption. This tool is only exposed to models with vision support. Images are downscaled to a maximum of 1 megapixel before being sent to the model.
 - Code Exec – Executes bash commands in the sandbox via the `code_exec` tool; returns exit code, stdout, and stderr.
 - Finish – Signals task completion and specifies which files to submit.
 - Abandon Task – Signals that the model does not believe it can complete the task, with a brief reason, instead of submitting files.

- For each task, a new E2B sandbox is initialized with the reference files associated with the given task and pre-installed with a range of relevant packages for the task set. We based the package collection on the disclosed environment from the original GDPval paper, expanded in v2 with additional dependencies (including a full TeX Live LaTeX toolchain and build tools).
 - View all 419 Python packages
 - View all 762 system packages
- System packages are the full pinned transitive closure resolved inside the Debian trixie base image, so most entries are dependencies of the packages we install directly
- We prompt the agent with an instruction interpolating the relevant task prompt, reference files, and finish tool details.
- Execution Limits:
 - The LLM is given 250 turns to complete the task. A single turn is defined as an assistant message and its tool calls (if any). As the model approaches the limit, it is notified of its remaining turn budget.
 - The model may end the run early via the Abandon Task tool where it does not believe it can complete the task, providing a brief reason instead of submitting files.
 - If the model exceeds 70% of its context window after completing a given turn, the agent asks it to summarize task state, completed work, current files, remaining steps, and important context, then clears earlier turn history while retaining the task prompt and the summary for continuation.

Task Submission System Prompt:

You are an AI agent completing a standalone professional task. Your job is to use the provided tools to produce the requested deliverables within 250 steps, then submit your work.

When you are done, call the `finish` tool as your final step with:

1. A brief summary of what you accomplished.
2. Absolute paths to every deliverable file.

If you have genuinely concluded that the task cannot be completed because required inputs are missing, a hard dependency is unavailable, or the request is incoherent, call the `abandon\_task\_finish` tool with a brief reason instead. Do not use it to escape difficulty.

You cannot interact with the user during the task. Make reasonable assumptions when needed and record them in your finish summary.

Task Submission Prompt:

Runtime

You are running in an isolated Linux sandbox. Use the `code_exec` tool to read, create, and modify files. Commands run as the non-root user `user` (UID 1000). Default working directory is `/home/user`.

Every command runs independently: no working directory, environment variable, or other shell state carries over from one call to the next. Prefer absolute paths for both files and commands, and do not navigate with `cd` across calls – a `cd` in one command is gone by the next, so relying on it leaves you silently operating in the wrong place. When a step genuinely needs a different directory, chain it into the same command (e.g. `cd /home/user/work && python build.py`).

A broad scientific-computing and document-processing stack is already installed, so confirm what is present before assuming a gap:

- Python 3.13 with the usual data stack (numpy, pandas, polars, scipy), plotting (matplotlib, plotly), the scikit-learn ML family, and document tooling (python-docx, python-pptx, openpyxl, PyMuPDF, pdfplumber, reportlab, weasyprint, Pillow, opencv), plus Playwright.
- System tools include LibreOffice, Pandoc, Tesseract, FFmpeg, ImageMagick, Ghostscript, TeX Live, OpenJDK, Chromium, jq, and git.
- Commands are terminated after 10 minutes. Keep them bounded, persist intermediate results to disk, and split long jobs into smaller steps.

Reference Files Location

(This section appears only when the task includes reference files.)

The reference files for the task are available in your environment's file system.

Here are their paths:

- [absolute path to each reference file]

Completing Your Work

In order to complete the task you must use the `finish` tool to submit your work. If you do not use the `finish` tool you will fail this task!

As a last resort if you really cannot make any meaningful progress, use `abandon_task_finish` with a brief reason instead of submitting files.

****Required in your finish call:****

1. A brief summary of what you accomplished
2. A list of ****ABSOLUTE file paths**** for the required output files (Do not submit folders).

Task

Here is the task you need to complete:

[task description]

Please begin working on the task now.

- Context Overflow: If the next model call (or the summarization request itself) exceeds the context window, the agent keeps unwinding earlier turns until summarization succeeds.
- Task Completion: To complete the task, the LLM must call the finish tool, providing a summary of the work done and the paths of the files it intends to submit. This tool can be used at any turn.
- Grading: We sample pairwise matches between model submissions in two stages:
 - Balanced sampling: We first sample each model diversely, balancing exposure across tasks, judges, and opponents, to seed initial ratings.
 - Active sampling: After the initial phase, we transition to Elo-informed sampling that prioritizes pairings between models with similar ratings to derive the most information per comparison. We maintain balanced exposure of tasks within each model throughout the process.
 - Submissions are randomly anonymized as Submission A and B to mitigate any model or position bias from the grader model.
 - Matches are graded by a panel of three frontier LLM judges from leading labs, each run at its default reasoning settings: GPT-5.6 Sol (medium reasoning), Gemini 3.8 Flash (high reasoning), and Claude Opus 5 (high effort). We sample between the judges for each comparison. The initial task, all reference files, and all submission files are parsed and provided as context to the judge.
 - Document-based files (.pdf, .docx, .pptx, .xlsx, etc.) are parsed as both text and as images. We extract .zip files and parse each individual file separately. For tasks containing audio or video files, the comparison is routed to Gemini 3.8 Flash, which handles these modalities natively. This context is embedded in a grading prompt that asks the judge to determine which of Submission A and B better responds to the task.
 - Final Scoring: Our final Elo score is a Bradley-Terry rating computed via maximum likelihood estimation from all pairwise comparisons (ties counted as half-wins for each side), anchored to DeepSeek V4.1 Flash (max) at 1600. The 95% confidence intervals are computed using the sandwich estimator to quantify rating uncertainty.

[AutomationBench-AA](#)

- Description: AutomationBench-AA is Artificial Analysis' run of Zapier's [AutomationBench](#). It tests whether models can complete realistic SaaS workflows that span multiple simulated business apps, using REST APIs as the tool interface.

- Paper: <https://arxiv.org/abs/2604.18934>
- Leaderboard: <https://zapier.com/benchmarks>
- Repository: <https://github.com/zapier/AutomationBench>
- Dataset:
 - We evaluate a private 657-task held-out split from AutomationBench dataset version 1.0.6
 - The tasks cover six business domains: Finance, HR, Marketing, Operations, Sales, and Support
 - They run in simulated app environments that include products such as Gmail, Google Sheets, Slack, Salesforce, Zendesk, Jira, and HubSpot
- Implementation:
 - We run each task once in the AutomationBench multi-turn environment, with a 50-turn cap. Models use the API toolset, discovering and calling the REST endpoints they need through structured tool calls
 - We classify each AutomationBench assertion as either an objective, which must be made true by the agent, or a guardrail, which initially passes and must not be broken by the agent
 - Objectives and guardrails are graded using programmatic checks on the final environment state. AutomationBench-AA does not use a separate LLM judge for grading
 - For the headline score, a task receives 0 if the model violates any guardrail. If no guardrails are violated, the task receives the percentage of objectives the model completed. Errored tasks also score 0
 - Each task belongs to one business domain, so domain breakdowns are mutually exclusive subsets of the task set. App breakdowns are not mutually exclusive: a task can involve multiple apps, so its objective and guardrail assertions may contribute to more than one app

<https://github.com/ArtificialAnalysis/Stirrup.git>
[ArtificialAnalysis/AA-Briefcase-Lite · Datasets at Hugging Face](#)
 from datasets import load_dataset

```
ds = load_dataset("ArtificialAnalysis/AA-Briefcase-Lite")
```

<https://arxiv.org/abs/2510.04374>
[openai/gdpval · Datasets at Hugging Face](#)
 from datasets import load_dataset

```
ds = load_dataset("openai/gdpval")
```

<https://github.com/zapier/AutomationBench.git>
<https://arxiv.org/pdf/2604.18934>